Day 12 – Print Subsequences

1️⃣ Subsequence kya hota hai?

👉 Subsequence = **array/string ke elements ko choose ya skip karna (order same rehta hai)**

Example

String = "abc"

Subsequences:

""
"a"
"b"
"c"
"ab"
"ac"
"bc"
"abc"

👉 Total subsequences = **2^n**

2️⃣ Core Idea (Important)

Har character ke paas **2 choice hoti hai**:

1. Include karo
2. Exclude karo

👉 Isi se recursion tree banta hai 🌳

3️⃣ Java Code – Print All Subsequences

```java
package Day_12;

public class Subsequences {
    public static void printSubseq(String str, int i, String curr){
        //Base case
        if(i == str.length()){
            System.out.println(curr);
            return;
        }

        //Include current character
        printSubseq(str, i+1, curr+str.charAt(i));

        //Exclude current character
        printSubseq(str, i+1, curr);

    }
    public static void main(String[] args) {
        String str = "abcd";
        printSubseq(str, 0, "");
    }

}
```

🔥 Dry Run (Important)

str = "ab"

Call Tree:

```
        ""
      /     \
    "a"      ""
   /  \    /  \
 "ab"  "a"  "b"   ""
```

Output:

ab
a
b
""

4️⃣ Time Complexity

O(2^n)

Kyuki har element ke liye 2 choices hain.

5️⃣ Important Variation

- **Print only non-empty subsequences**

```
if(curr.length() > 0){
    System.out.println(curr);
}
```

- **Count subsequences**

```
public static int countSubseq(String str, int i){

    if(i == str.length()) return 1;

    int include = countSubseq(str, i+1);
    int exclude = countSubseq(str, i+1);

    return include + exclude;
}
```

Problem 1 – Print All Subsequences of String

💡 Concept

Har character ke paas 2 choice:

- include karo
- exclude karo

✅ Code

```
public class AllSubsequences {

    public static void printSubseq(String str, int i, String curr){

        if(i == str.length()){
            System.out.println(curr);
            return;
        }

        // include
        printSubseq(str, i+1, curr + str.charAt(i));

        // exclude
        printSubseq(str, i+1, curr);
    }

    public static void main(String[] args) {
        printSubseq("abc", 0, "");
    }
}
```

Problem 2 – Count Total Subsequences

💡 **Concept**

- Include + Exclude
- Total = 2^n

✅ **Code**

```
public class CountSubsequences {

    public static int countSubseq(String str, int i){

        if(i == str.length()) return 1;

        int include = countSubseq(str, i+1);
        int exclude = countSubseq(str, i+1);

        return include + exclude;
    }

    public static void main(String[] args) {
        System.out.println(countSubseq("abc", 0));
    }
}
```

Output:

8

Problem 3 – Print Only Non-Empty Subsequences

💡 **Concept**

Empty string skip karna hai

✅ **Code**

```
public class NonEmptySubseq {

    public static void printSubseq(String str, int i, String curr){

        if(i == str.length()){
            if(curr.length() > 0){
                System.out.println(curr);
            }
            return;
        }

        printSubseq(str, i+1, curr + str.charAt(i));
        printSubseq(str, i+1, curr);
    }

    public static void main(String[] args) {
        printSubseq("abc", 0, "");
    }
}
```

Problem 4 – Print Subsequences of Array

💡 **Concept**

String ki jagah array use karna hai

✅ **Code**

```java
package Day_12;

public class ArraySubsequence {

    public static void printSubseq(int[] arr, int i, String curr){

        if(i == arr.length){
            System.out.println(curr);
            return;
        }

        // include
        printSubseq(arr, i+1, curr + arr[i] + " ");

        // exclude
        printSubseq(arr, i+1, curr);
    }

    public static void main(String[] args) {

        int[] arr = {1,2,3};
        printSubseq(arr, 0, "");
    }
}
```

Problem 5 – Subsequence Sum = Target

💡 **Concept (Very Important 🔥)**

👉 Har element ke paas 2 choice:

- include karo → sum badhega
- exclude karo → sum same rahega

✅ Code

```
1. public class SubsequenceSum {

    public static void findSubseq(int[] arr, int i, int sum, int target, String curr){

        if(i == arr.length){

            if(sum == target){
                System.out.println(curr);
            }

            return;
        }

        // include element
        findSubseq(arr, i+1, sum + arr[i], target, curr + arr[i] + " ");

        // exclude element
        findSubseq(arr, i+1, sum, target, curr);
    }

    public static void main(String[] args) {

        int[] arr = {1,2,1};
        int target = 2;

        findSubseq(arr, 0, 0, target, "");
    }
}
```

🔥 Dry Run (Important)

arr = [1,2,1]
target = 2

Valid subsequences:

2
1 1

🎯 Day 12 Final Summary

Problem	Concept
All subsequences	include/exclude
Count subsequences	2^n
Non-empty	skip empty
Array subsequence	same logic
Target sum	include + sum